

EGA - Carga Electrónica Programable

Gallo Acosta, Alejandro Martín

alejandromgallo1@gmail.com

1. INDICE

1. INDICE	1
2. ABSTRACT	1
PALABRAS CLAVE:	1
3. FUNDAMENTACION	1
4. OBJETIVOS	1
5. ALCANCE PROPUESTO	2
6. DIAGRAMA DE SOFTWARE	2
7. DIAGRAMA DE TAREAS EGA	2
8. DIAGRAMA DE CONEXIONES	2
9. LISTADO DE COMANDOS	2
10. CONCLUSIONES	2
11. REFERENCIAS	2
12. ANEXOS	3
ANEXO I – DIAGRAMA DE SOFTWARE	3
ANEXO II – DIAGRAMA DE TAREAS	4
ANEXO III – DIAGRAMA EN BLOQUES	5

2. ABSTRACT

Este documento tiene como objetivo explicar el desarrollo e integración de un sistema de comunicación entre el sistema embebido con sistema operativo en tiempo real (RTOS) desarrollado previamente en la EGA y un sistema operativo de propósito general GPOS (EGB), mediante el diseño e implementación de un driver de kernel en Linux. El trabajo consiste en la creación de un módulo de kernel que implementa una interfaz de tipo char device y que, a su vez, utiliza la capa serdev para la comunicación serie sobre UART con el dispositivo embebido. El driver desarrollado permite el envío y recepción de comandos desde el espacio de usuario (User Space), tanto en una aplicación de comandos CLI como en una aplicación web GUI. El driver fue programado sobre el IDE Visual Studio Code en lenguaje C, y la aplicación de User Space programada en Python, HTML, CSS y Javascript. El sistema permitirá la lectura y modificación de parámetros del setpoint, resistencia y corriente de forma remota, como así también la visualización en tiempo real de la corriente en el tiempo.

PALABRAS CLAVE:

Carga electrónica, PID, Kernel, Driver, Char Device, Serdev

3. FUNDAMENTACION

La necesidad de interconectar sistemas de tiempo real con sistemas de propósito general es un aspecto fundamental en sistemas embebidos modernos, especialmente en aplicaciones industriales, automotrices y de control.

En este contexto, la etapa EGB se centra en el desarrollo de un **driver de kernel** que permita una comunicación confiable entre un sistema basado en Linux y una plataforma embebida que opera bajo un RTOS. A diferencia de soluciones de alto nivel, la implementación a nivel de kernel permite un mayor control de los recursos, mejor gestión de concurrencia y manejo directo del hardware.

El driver desarrollado integra dos mecanismos principales:

- Un **dispositivo de caracteres (*char dev*)** que expone una interfaz en `/dev/egb` para aplicaciones de usuario.
- Una capa de comunicación serie mediante **serdev**, que gestiona la UART a nivel kernel y permite una comunicación eficiente y de baja latencia.

4. OBJETIVOS

- Implementar un **módulo de kernel** que registre un dispositivo de caracteres accesible desde el espacio de usuario.
- Desarrollar la **comunicación UART a nivel kernel** mediante la infraestructura **serdev**.
- Desarrollar una **interfaz de línea de comandos (CLI)** y una **interfaz gráfica de usuario (GUI)** que permita setear y tomar parámetros de la EGA.
- **Habilitar la comunicación SSH** estableciendo un canal seguro para la interacción remota entre la PC del usuario y la Raspberry Pi.

5. ALCANCE PROPUESTO

- Implementar comandos **GET de parámetros de setpoint, logs y variables PID.**
- Implementar comandos **SET de parámetros de setpoint y variables PID.**
- **Gráfico de corriente en el tiempo** en interfaz visual.

6. DIAGRAMA DE SOFTWARE

Para ver el diagrama del software, ver **Anexo I.**

7. DIAGRAMA DE TAREAS EGA

Para poder comunicar la EGA con el kernel mediante UART tuvieron que realizarse modificaciones en el desarrollo del código en FreeRTOS.

Para ver el diagrama completo de tareas, ver **Anexo II.**

8. DIAGRAMA DE CONEXIONES

Para ver el diagrama de conexiones completo, ver **Anexo III.**

9. LISTADO DE COMANDOS

COMANDOS SET

- **SET KP “valor”**
- **SET KI “valor”**
- **SET KD “valor”**
- **SET T “valor en ms”**
- **SET R “valor resistencia”**

COMANDOS GET

- **GET KP**
- **GET KI**
- **GET KD**
- **GET SP** → *Setpoint*
- **GET LOG** → *Log completo de la lectura*
- **GET R** → *Resistencia actual*
- **GET I** → *Corriente*

10. CONCLUSIONES

El desarrollo de la EGB permitió consolidar y expandir las capacidades del sistema implementado en la EGA, logrando una integración completa entre un sistema embebido basado en RTOS y un sistema operativo de propósito general (GPOS) mediante la implementación de un driver de kernel en Linux.

La creación del módulo de kernel tipo *char device* junto con el uso de la infraestructura *serdev* para la comunicación UART permitió establecer un canal de comunicación robusto y confiable entre la Raspberry Pi 4 y la carga electrónica programable (EGA). Este enfoque a nivel de kernel demostró ser eficiente y controlado, sin traer mayores problemas a la hora de la implementación y las pruebas.

El desarrollo de la aplicación CLI y una interfaz web GUI permitió validar el funcionamiento del driver desde el User Space, facilitando la interacción con el sistema tanto de forma local como remota. La interfaz visual fue probada desde diferentes PC y se pudo comunicar de forma exitosa y a alta velocidad. La posibilidad de enviar comandos, leer variables del sistema y visualizar datos en tiempo real demostró la correcta integración entre las capas de hardware, kernel y aplicaciones de usuario.

En la primer instancia de implementación se detectaron posibles errores en los primeros comandos SET/GET desde la interfaz gráfica y de comandos, pero luego pudieron resolverse haciendo ajustes en el driver del *char device*, ya que quedaba leyendo un valor anterior.

El gráfico de la corriente en función del tiempo fue clave para poder realizar ajustes en el control PID sin necesidad de conectar un osciloscopio. Este gráfico permitió ver la velocidad de respuesta del sistema, pudiendo hacer actualizaciones cada aproximadamente 20ms.

En conclusión, el sistema final desarrollado cumple satisfactoriamente con los objetivos planteados para la EGB, habiéndose superado el alcance pensado previamente.

11. REFERENCIAS

[1] R. Love, *Desarrollo del kernel de Linux*, 3.a ed. Boston, MA, EE. UU.: Addison-Wesley, 2010.

[2] Linux Foundation, “Documentación oficial del kernel de Linux,” 2024. [En línea]. Disponible en: <https://www.kernel.org/doc/>

[3] Raspberry Pi 4b specifications. Disponible en: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>

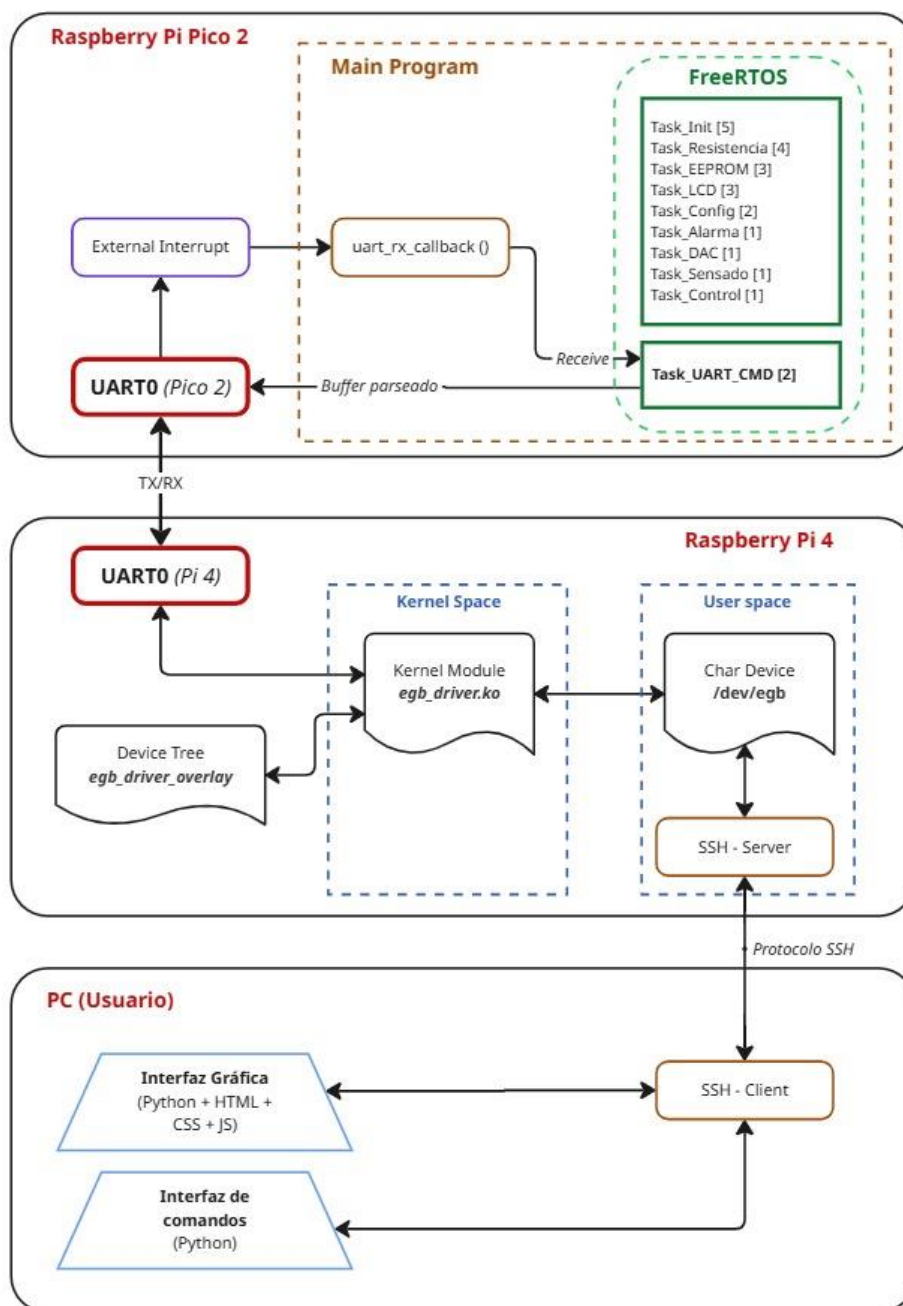
[3] FreeRTOS, “Documentación del kernel de FreeRTOS,” 2024. [En línea]. Disponible en: <https://www.freertos.org>

[4] Raspberry Pi Foundation, “Hoja de datos del RP2040,” 2021. [En línea]. Disponible en: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>

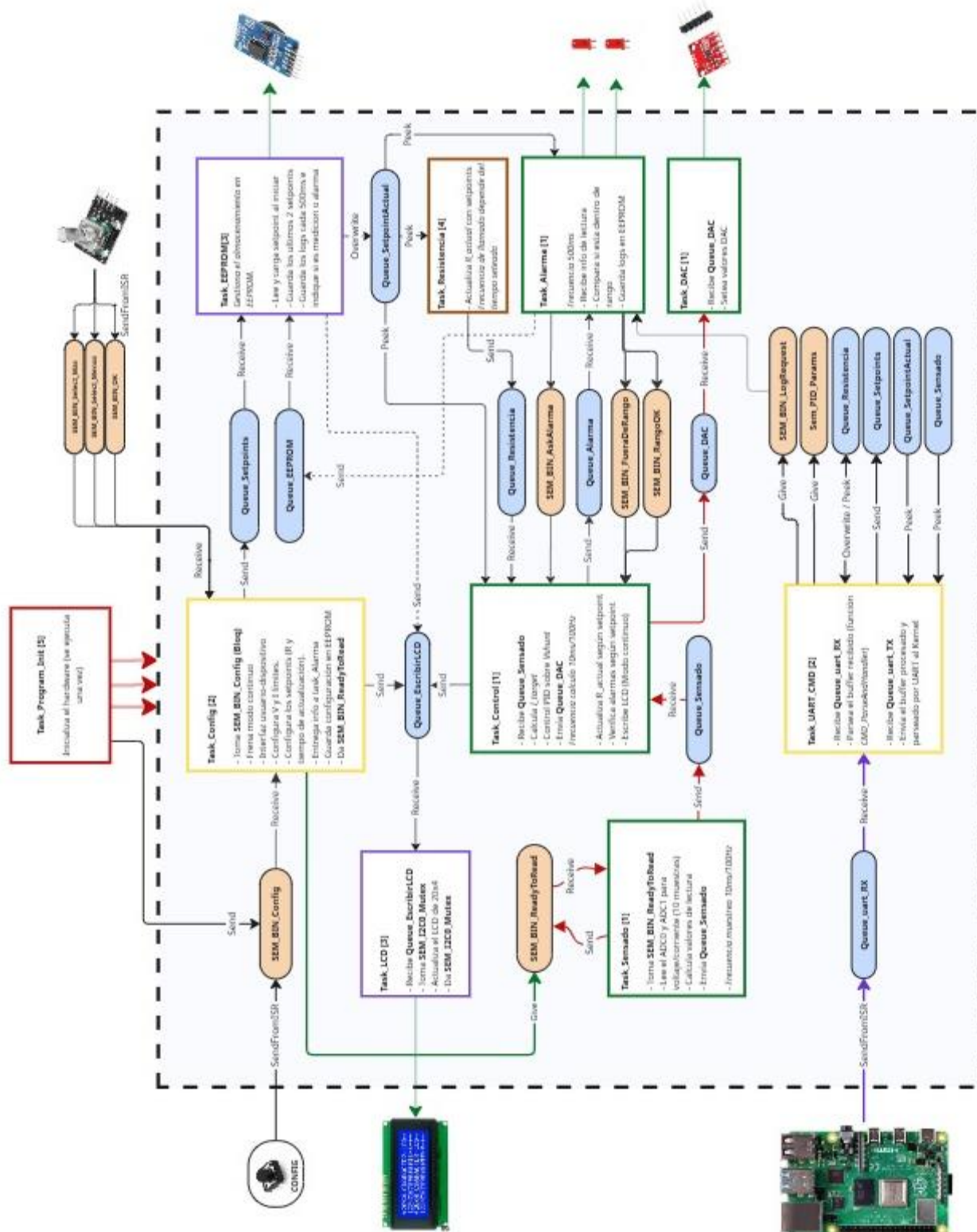
[5] Flask, “Documentación del framework Flask,” 2024. [En línea]. Disponible en: <https://flask.palletsprojects.com>

12. ANEXOS

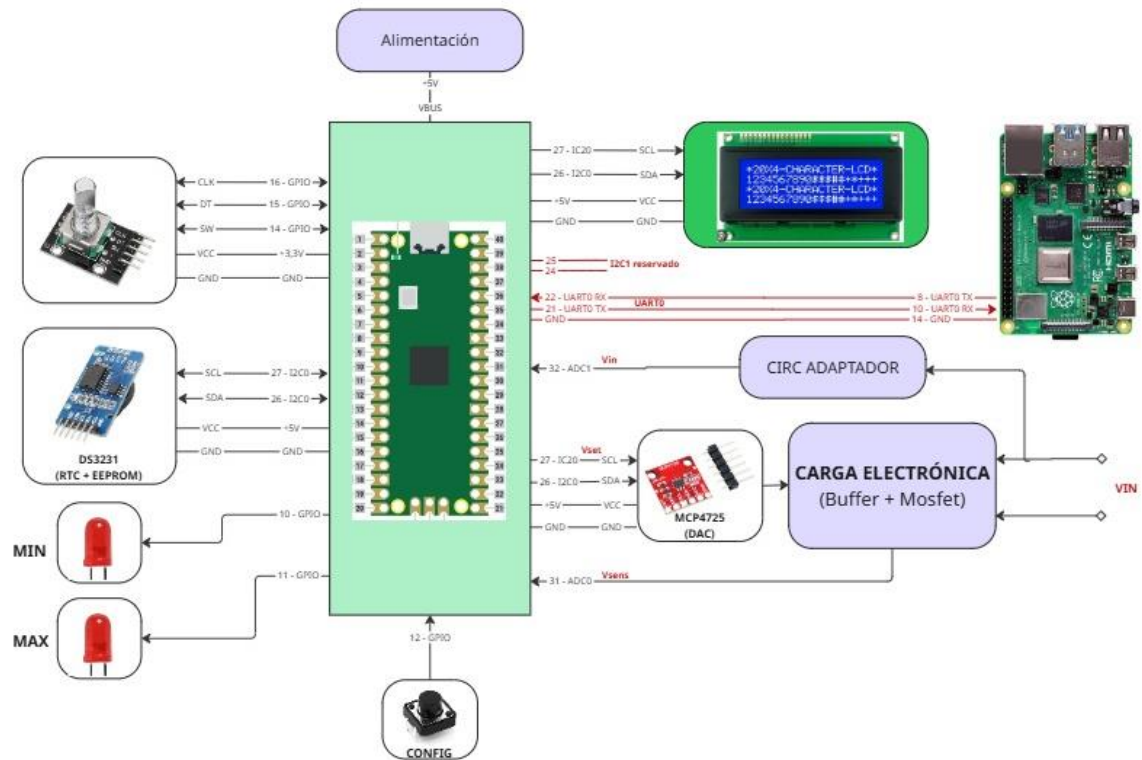
ANEXO I – DIAGRAMA DE SOFTWARE



ANEXO II – DIAGRAMA DE TAREAS



ANEXO III – DIAGRAMA EN BLOQUES



ANEXO IV – INTERFAZ GRÁFICA



